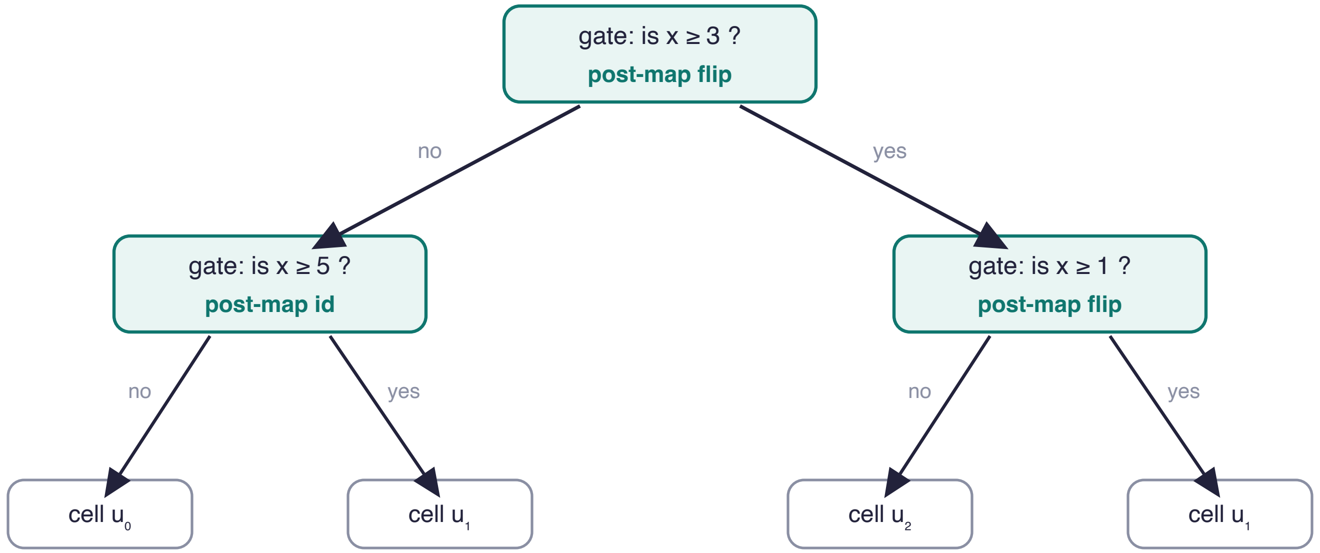
A PROGRAM IS A TREE · drawn from the construction



one candidate = one finite tree: a gate picks a child by a threshold on the input, the chosen child answers, the node's post-map finishes. The compute family is constitutive: an elementary cell at every base, a finite post-map at every node.

free selection is forbidden by the machine's own theorem: with unrestricted selectors, one step reaches every function, so candidates must be trees (data).

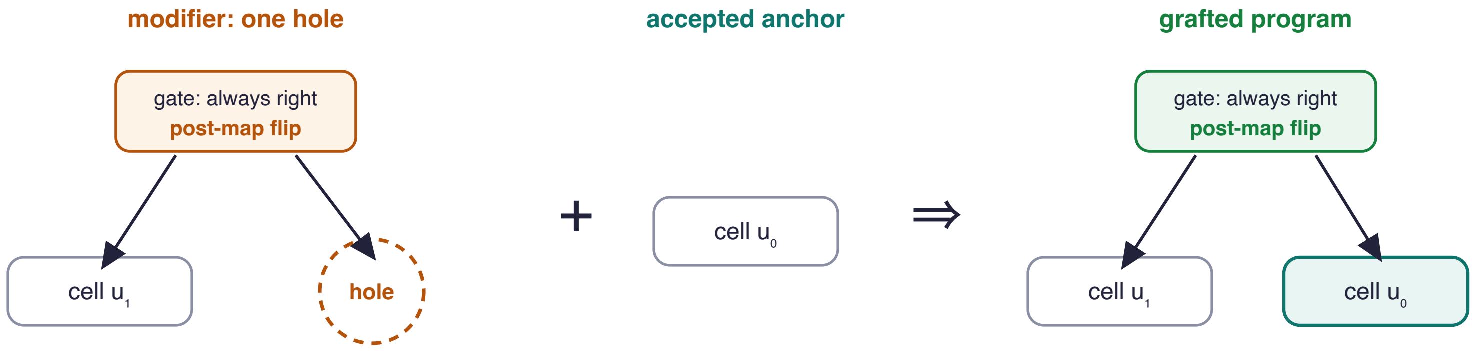
THE TOWER · strict at every rung, priced at every rung

capacity ≤ 2^(depth+1), proved at every rung

every rung strictly larger, on the counting world and on the real-vector world; the union over all depths never exhausts the function space. Retrieval over depth-sorted candidates returns a least-depth fit: built-in simplicity bias.

the machine of page one runs verbatim over this forest: same retrieval, same laws.

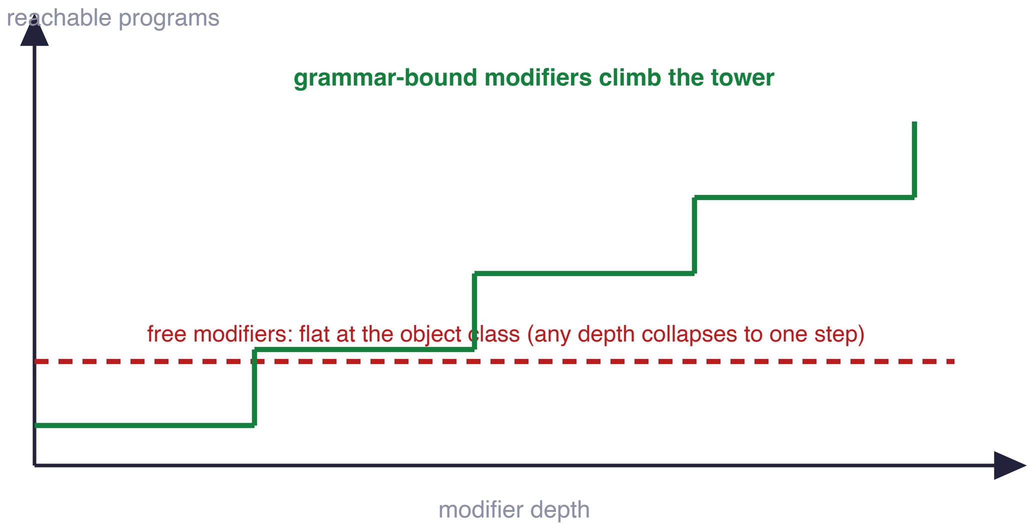
MODIFY BY GRAFTING · the first rung climbs, in one picture



grafting is part of the grammar: a plugged modifier IS a tree, so modified programs inherit every law. Here the gate always routes to the graft and the post-map flips it: the grafted program disagrees with its anchor on every input. One modifier, one rung.

retrieval over modifiers = grafting: the accepted anchor enters every candidate by the hole.

FREE IS FLAT, BOUND CLIMBS · the dichotomy



embedding within two rungs; strict climb proved at stride three;
in the limit the graft loses nothing: the bound ladder fills the whole tower.

certificates chain at every storey (page two); the per-storey capacity product prices execution.